De un consolidado a una consulta clara

Estructura, criterios y mantenimiento de la solución

La solución transforma el consolidado de indicadores en una página de consulta y un espacio protegido de actualización. Conserva la identidad visual de La Perla y usa lenguaje sencillo.

La información de partida

El histórico inicial corresponde al consolidado 2026 de enero a septiembre. Los meses siguientes quedan pendientes hasta que se registren datos. Las filas de clínicas operadoras externas, como CES y FUNCUAN, se excluyen. No se agregaron años ficticios a la publicación.

Lectura de los archivos

ExcelJS lee archivos .xlsx en el navegador. El importador reconoce las hojas de los meses, las etiquetas de indicadores y las columnas semanales. El año se solicita al usuario porque un encabezado puede conservar un año anterior. Antes de guardar se muestran los meses reconocidos y posibles diferencias.

Criterios de cálculo

Las actividades se suman sin volver a agregar sus desgloses. La población alojada y apta usa el último conteo. Los vacíos se mantienen como desconocidos. Los promedios de alimentos usan meses completos; las comparaciones entre años pueden limitarse a meses completos comunes. Las notas distinguen cifras parciales y diferencias del archivo.

Un ajuste documentado

En julio, una cantidad de alimento para perros adultos estaba guardada como texto numérico: 4.369,5 kg. El total impreso del Excel la omitía. El panel toma la suma semanal de 20.908 kg y deja el aviso de revisión; no modifica el archivo original. Esto evita que una fórmula incompleta arrastre un consumo menor.

Semanas fieles al origen

El filtro semanal usa el número y la etiqueta de la semana del mes. Conserva los números aunque existan huecos entre registros. El detalle territorial se mantiene mensual. No se inventan distribuciones semanales ni se interpretan los conteos como número de animales únicos.

Abrir el dashboard de La Perla

Qué hace cada parte

Cómo fue construido

La página de consulta y el servicio que guarda los datos cumplen funciones distintas. Esta separación permite publicar la vista en GitHub Pages sin trasladar la clave al navegador.

Interfaz: HTML, CSS y JavaScript

app/index.html contiene la estructura y el histórico inicial. Las hojas de estilo definen colores, tipografía y adaptación de controles. Los gráficos se dibujan como SVG. week-filter.js calcula la selección semanal; weekly-core.js combina el Excel base con las correcciones. multiyear-client.js coordina años, comparación, importación y administración.

Informes y archivos

report-pdf.js usa jsPDF para construir los informes con textos y gráficos en el propio PDF. El logo se incluye en el archivo. excel-import.js interpreta los Excel y comunica los avisos. Los manuales se generan aparte con Python y ReportLab; su contenido editable está en manuales/manuales.json.

Servicio de datos

worker/index.js atiende las solicitudes. D1 guarda las semanas, los datos normalizados por año, las revisiones y las sesiones. R2 conserva los Excel originales importados. db/schema.ts describe las tablas y drizzle/ conserva las migraciones. Una misma combinación año/mes/semana no crea duplicados.

Protección real al guardar

El servidor comprueba la identidad administradora recibida de Sites, la clave secreta, la sesión vigente y el origen de la petición. La cookie de sesión no se puede leer desde JavaScript. El límite de intentos y las revisiones ayudan a controlar accesos y conflictos. La clave real no está incluida en el paquete ni en los manuales.

GitHub Pages conectado

docs/ contiene la versión estática. config.js señala la URL pública de consulta y la administración. La lectura de /api/data permite acceso desde otro dominio sin enviar cookies. La edición abre el sitio protegido y no se ejecuta desde Pages. El servicio actual debe mantenerse activo; no se ha migrado la base de datos.

Mantener y verificar

Cómo fue construido

El paquete incluye el código y una versión lista para publicar. Los registros vivos y los archivos importados siguen en el servicio; el ZIP no sustituye una copia de seguridad de esos datos.

Cambios de diseño

Con Node.js 24, ejecuta npm ci una vez para instalar las versiones del archivo de dependencias. Edita app/ y ejecuta npm run build:pages. Luego sube el código y docs/ al repositorio. La publicación desde main y /docs no recompila app/ por sí sola. No guardes cambios únicamente en el HTML generado.

Cambios en el servicio

Requieren desplegar el Worker y sus migraciones en un entorno compatible. No basta con subir worker/ a GitHub Pages. Si se reemplaza Sites, se debe implementar una identidad administradora confiable antes de permitir escrituras. Las cabeceras de identidad no deben aceptarse directamente desde visitantes de Internet.

Qué se comprobó

La importación se contrastó con 38 indicadores y 9 meses del Excel de referencia. Las pruebas verificaron edición, persistencia, duplicados, conflictos de revisión, sesiones y restricciones de escritura. Se comprobó el filtro semanal con meses de cuatro, cinco y seis semanas, especies, semanas ausentes y cambios de periodo. Los datos de prueba se mantuvieron fuera de producción.

Alcance de la revisión visual

Los PDF se renderizaron a imágenes y se revisaron por página. Los controles tienen reglas para pantallas pequeñas y pruebas de funcionamiento. No se dispuso de un navegador móvil para certificar su apariencia real. Después de publicar en GitHub, revisa al menos un teléfono y una pantalla de escritorio.

Para una futura migración completa

Además del código, prepara una exportación de los datos y de los Excel almacenados, un proveedor de identidad, una base de datos y almacenamiento de archivos. Prueba lectura, corrección, recuperación y permisos antes de cambiar la dirección del servicio. Esta entrega mantiene la conexión existente y no incluye esa migración.